The shape of the answer. An append-only archive of station readings, written through a durability gate that only acknowledges after `fsync`, stored as one self-describing log per station, compacted into rollups so history stays cheap forever, and read through three cheap primitives: tail reads, ETag revalidation and recency-weighted correlation. No database, no cluster: the whole thing is two static binaries and a Next.js front end, because the machines are cheap and expected to disappear.

What I built, and why

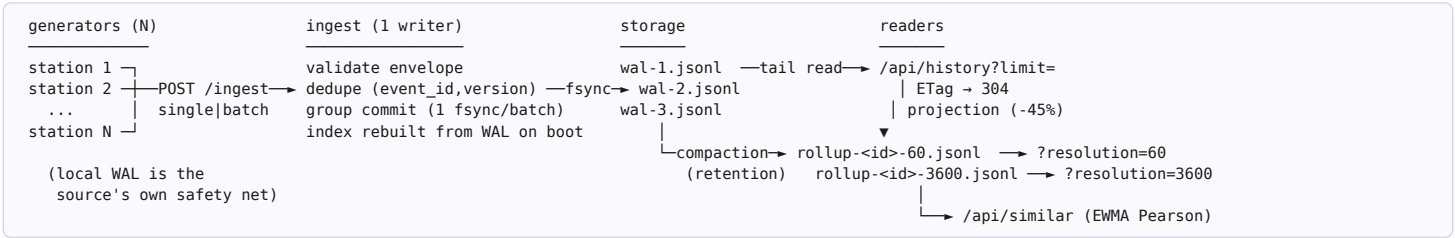
1. Generators that produce the pathologies, not just the happy path. One process per station, stdlib Python. It emits regular readings and then misbehaves on purpose: reassembled out of order, duplicated byte-for-byte, corrected with a version bump, timestamped minutes in the past, silenced for outages, and eventually killed forever (`--die-after`). Ids are `station:run:seq`, versioned. This was the decision that later saved the project: without run-scoped ids, a relaunch silently reuses ids and shadows its own history.

2. Ingest: acknowledgement is a durability claim. `POST /ingest` accepts one record or a batch, validates the envelope, and answers only after the bytes are on stable storage. One goroutine owns all writes; concurrent requests share a single write+`fsync` per station per batch (**group commit**). Deduplication is on `(event_id, version)`, highest version wins, so a retry after a crash is a no-op returning `200` with `duplicates: 1`. The dedupe index is not a database — it is rebuilt from the WALs at startup, so it cannot drift from the data. Records are validated on the envelope only and stored **byte-for-byte**, so adding a field later needs no migration and no writer redeploy.

3. Storage: one log per station, plus rollups. `wal-<id>.jsonl` is the source of truth. A maintenance pass folds anything older than a retention window into per-minute and per-hour aggregates (`rollup-<id>-60.jsonl`, `-3600.jsonl`) and then prunes the raw log. The cutoff is aligned to whole buckets and rollups are written and fsynced **before** the rewrite, which makes a crashed compaction idempotent: re-running re-folds identical whole buckets and produces identical aggregates.

4. Read path: make the common query cheap, and the repeat query free. `limit` queries walk backwards from EOF instead of parsing the file; the serialised response body is cached per `(station, limit)` and invalidated by size+mtime; responses carry a strong `ETag`, so a polling client gets `304` with no body. Coarse ranges are served from rollups (`?resolution=3600`). Records are projected to the five fields consumers actually use.

5. Similarity: recency-weighted correlation. Per-station means are bucketed to 30 s and compared with EWMA Pearson — $\bar{x} \leftarrow \lambda\bar{x} + (1-\lambda)x$, same for the covariances — over buckets both stations actually observed. λ is explicit and returned in the response (`λ=0.999` ≈ last 1000 buckets ≈ 8 h), because "similar" without a timescale is not a question. Batch Pearson remains available as the reference implementation, and the benchmark measures both.



Decisions I would defend

decision	chose	gave up	why it was worth it
storage	one append-only log per station	cross-station transactions	the only join is by station; a log is crash-safe and human-readable
ack semantics	sync before responding	5.6 ms per single record	the promise is what the brief actually asks for
write throughput	group commit	a few ms of added latency under concurrency	180 → 24,000 records/s with the same guarantee
dedupe	rebuild index from the WAL	startup scan time	no second store to drift; recovery is self-evident
schema	validate envelope, store bytes	no enforced typing of payload	new fields need no migration or redeploy
long ranges	rollups + retention	exact raw points past retention	219 MB → 75 MB; a year of hours in 679 KB
reads	tail read + cached body + ETag	cache is per process	300× faster; repeated polls cost 1.6 ms
similarity	EWMA Pearson, λ explicit	phase shift, non-linearity	interpretable, parameter-light, honest about timescale

Numbers

Seeded a synthetic year — 1,136,068 readings, 219 MB, of which station 1 alone is 516,698 readings (103.7 MB) — and ran old and new against the **same** server and data. The old handler is preserved as a gated `/api/history-legacy`, so the comparison is reproducible rather than remembered.

path	p50	p95	payload	why
`history` (old)	1147 ms	1339 ms	100 KB	parses 104 MB per request
`history limit=500`	2.7 ms	6.8 ms	55 KB	tail read + cached body
`history` 304	1.6 ms	3.6 ms	0 B	ETag revalidation
`history` cold	4.0 ms	9.2 ms	55 KB	cache off: 1.7 ms server
`history` load	12.9 ms	39.0 ms	55 KB	565 req/s, 8 workers
`similar` EWMA	2.2 ms	4.8 ms	303 B	pairwise signed sums, cached
`similar` batch	20.0 ms	24.5 ms	261 B	recomputes arrays

Write path, fsync enabled: a single acknowledged record costs **5.6 ms** (that is the durability bill); group commit lifts the same guarantee to **24,075 records/s** at 16 workers. Compaction turns the 219 MB year into **75 MB** with a year of hourly aggregates in 679 KB per station; a spot-checked hour bucket matched the raw data exactly (`n=61`, mean to 1e-9, min/max identical).

The assumption I am least sure of

That one machine's local disk is an acceptable durability boundary. Everything above treats a successful `fsync` on a single node as "safe", and the acknowledgement is a promise made on that basis. I believe it is the right call for cheap, disposable workers — it is the cheapest thing that can honestly say "acknowledged data is never lost" — but it is one disk, one kernel, one power supply.

If it is wrong, the failure is quiet and total. A node that dies permanently takes every acknowledged-but-unreplicated record with it, and nothing in the system notices: there is no quorum, no second copy, no fencing, so a restarted node can also accept writes that another node still believes it owns. Consumer SSDs make this worse, not theoretical: a drive that acknowledges `fsync` while data still sits in a volatile cache breaks the promise underneath the code that made it. The fix is not clever — replicate to two peers and acknowledge on the second, or hand durability to object storage — but it changes the shape of the system (leader election, or at least idempotent replay with fencing tokens), and it would cost the simplicity that makes the rest of this defensible.

The runner-up, less dramatic: I assume per-station volume stays in the 10^2–10^3 MB range and that ingest is not adversarially bursty. At 100× the rate, group commit depth and the single-writer goroutine become the bottleneck before the read path does.

What I learned, and the part worth teaching

Identity is harder than performance. The expensive-looking problem was latency; the one that actually broke correctness was an id. Because the generator restarted `seq` at 1 on relaunch while appending to the same log, 2,715 of 4,997 `event_id`s were reused. Dedupe kept the first sighting, so a full-file read and a tail read returned **different rows for the same query** — and every benchmark still looked fine. Two fixes, both cheap: stamp the run into the id, and break version ties deterministically on `emitted_at` so the answer never depends on how much of the file you happened to read. The lesson I would teach from this: **an id is a claim about the world, not a serialisation detail.** Dedupe silently converts a modelling error into data loss, and it does so while the dashboards stay green.

Durability is a batching problem, not a speed problem. `fsync` costs what it costs; the only lever is how many records share it. One record per sync: 180/s. Batched: 24,000/s, same guarantee. That ratio is the whole design of the write path.

And the one I keep coming back to: a similarity metric is a preprocessing decision. Pearson over the seeded year scored **0.92 between every pair**, because every station shared the same year-long ramp — the number was measuring the calendar. Differencing the series dropped the same pair to **0.44**. Switching Pearson for EWMA Pearson changed **which** data counted, not what was being measured; only detrending did that. So "which metric?" is the wrong first question. "Which invariance do I want, and what have I removed before measuring?" is the right one, and the answer has to be written down, because the code cannot tell you whether 0.92 means "these stations behave alike" or "these stations both went up".

Code

`generator.py`, `start-generators.sh` — station simulator with injected chaos
`ingest/` — durable, idempotent write path + compaction (Go, tested)
`web/` — history/similar APIs, rollup reads, Next.js dashboard
`web/bench.go`, `seed_data.py` — reproducible benchmarks at 1-year scale
`web/lib/history.test.ts` (12 tests), `ingest/main_test.go` (13 tests)

How to check any of this

- `./start-generators.sh 10` — ten stations that misbehave on purpose
- `cd ingest && go test ./...` — 13 tests: dedupe, versions, ack-implies-durable, crash recovery, 1,200 concurrent posts, compaction idempotence
- `cd web && npm test` — 12 tests against the real read path: canonicalisation, tail == full, EWMA, rollups
- `cd web && go run bench.go --station 1 --limit 500` — old vs new, same data, same server
- `python3 seed_data.py --days 365` — regenerate the 219 MB year used above
- `ALLOW_LEGACY=1` — re-enables the pre-optimisation handler the benchmark compares against